
WHEN DO LANGUAGE SERVERS HELP CODING AGENTS?

Ian Barber

July 2026

ABSTRACT

Coding agents are increasingly given language-server tooling on the assumption that better code intelligence makes a better agent. We explored that assumption against a capable baseline of `grep`, ranged reads and a shell, across 7 models. Making LSP tools available did not change anything: agents largely declined to use the operations they were offered. Gains required a prompt, training, or a gate that blocked the work. Go-to-definition helped when resolving the receiver’s type required a retrieval step rather than being already in context, and only when the agent was instructed to use it. Definition spans saved tokens only when they replaced the file read, which is workload dependent: on one suite models reread on 35 of 36 injected spans. Merely adding prompting to say the span was complete removed reads in just 2 cases: adjusting the behavior required fine-tuning. On 12 seeded defects, a type checker gating submission took accepted correct outcomes from 1 to 11. Earlier delivery of type errors made no difference, and actively guiding the model to respond to earlier diagnostics performed worse than no feedback at all.

If you use a coding agent: run your type checker at the end of the turn as a blocking gate; add navigation tools only as the codebase gets more complex, with an instruction to use them, and expect the most gains when resolving a type means reaching into a stub, a generated client or a compiled boundary; expect to save tokens only once the agent stops reading the file, so measure that before counting the savings.

1 Introduction

A language server answers questions about code. An agent with `grep`, ranged reads and a shell already answers most of them itself, so what a server adds depends on whether it answers a question the agent cannot (IF), whether its answer is cheaper in tokens (FORM), and whether it arrives at a better moment (WHEN). Prior work on completion and test generation compares against retrieval rather than against an agent that can look things up for itself; work on agents compares whole interfaces at once. We hold the harness and the model fixed and add or withhold one operation at a time, against a baseline of `grep`, ranged reads and a shell.

IF. Typed Holes [2] and LSPRAG [3] push types and definitions into context on the premise the information is missing.

FORM. CodeStruct [6] argues structured reads and edits use fewer tokens than whole-file handling.

WHEN. STALL+ [7] finds static-analysis value varying with integration phase; CoCoGen [1] checks a coherent draft before retrieving context to repair it.

2 Method

Most experiments ran in a custom harness: a fixed loop of six actions over a Python workspace (`grep`, ranged read, whole-file read, line edit, run tests, submit), and each experimental arm added or withheld one semantic operation. The definition operation in the retrieval experiments was a static AST resolver validated for equivalence against Pyrefly. Live Pyrefly served the dispatch lookup arms directly. A confirmation study ran mini-swe-agent [9, 11] with Claude Sonnet 4.5 on 3 SWE-bench [5] tasks (sympy, astropy, sphinx).

Note: We did not use an MCP server, skill or editor plugin.

Suite	N	Source	Temp x seeds	Isolates
Dispatch ladder	15 tasks x 3 variants	Synthetic; typed receiver, ~10 same-named overrides (one buggy); type moves from call-site annotation to construction site to factory indirection	0 x 1	Lookup when the type is already in context
Hidden-source dispatch	15 tasks, 450 rollouts	Same suite, with application and test source left on disk but not pasted into the prompt and the receiver construction redacted in the quoted asserts	0.7 x 5	Lookup when the type costs a retrieval step
Whole-file baseline	11 tasks x 4 seeds x 3 models	Constructed misuse over vendored toolz 1.1.0 and more-itertools 11.1.0	0.7 x 4	Span cost against a whole-file read
Text-retrieval baseline	11 tasks	As above	0 x 1	Span cost against grep with ranged reads
Definition spans	12 instances per arm	Synthetic arithmetic overrides; the delivered span is mechanically verified to contain the fix	0 x 1	Whether the span substitutes for the read
Delivery timing	14 tasks x 6 arms, 168 rollouts per arm	Synthetic single-file authoring; no feedback, 2 batched and 3 live delivery points	0.7 x 12	When a diagnostic should arrive
Checker grid	12 defect/clean pairs	Revision task: model reviews a pre-written draft carrying a seeded defect that passes the visible test; clean controls are validated gold	0 x 1	Timing of one diagnostic
Shell-agent case study	3 SWE-bench tasks	sympy, astropy and sphinx, through off-the-shelf mini-swe-agent with Claude Sonnet 4.5 and an AST-backed definition command	one seed, 60-step cap	Whether the constructed findings show up in a real agent on real repositories

Local models were Qwen2.5-Coder-7B, Qwen3.5-27B and Qwen3.6-27B; Claude Sonnet 4.5, DeepSeek v3.1, GLM-4.6 and GPT-5.6 “Luna” ran through an API. Most runs were temperature 0.7 with repeated seeds; some held-out retests ran at temperature 0. Workspaces are small enough to read any file completely; we did not test dynamic behavior, wrong annotations and Any-heavy boundaries. On real repositories, Codebase-Memory [10] reports the same shape from the other side: a file-exploration agent answered better than a structured graph tool, 92% against 83%, at roughly ten times the tokens. Code and artifacts: <https://github.com/ianbarber/lsp-for-llms>.

Measures:

- **Resolution:** fixed the intended target and passed a held-out test written against the specification, separate from the visible one.
- **Cost:** total tokens, input plus output, over the whole trajectory.
- **Election:** how often the agent chose an offered semantic operation.
- **Substitution:** reads of the defining file after a span was delivered.

Checker experiments also report wall time. Measurements are per-task; rollouts are grouped per task to get observations for intervals.

3 Results

3.1 IF: does delivering the semantic information change what the agent finds?

In the *dispatch ladder* suite all source was provided in the initial prompt, and the variants differed only in how far the type sits from the call site: on it, at the construction site, or behind a factory. The agent grepped 0.3 times per task and re-read source it had already been given in the prompt, navigating to the buggy override on the types available to it: text search and defn resolved equally at every level.

For tokens, cost was flat too: 1.03x text search when lookup was merely available, 0.96x when prompted, 0.94x to 1.06x across variants. The difference was substitution: the prompted agent swapped the span for its file read, resulting in 0.07 whole-file reads per task against 1.00, while the unprompted agent made both calls (0.80 lookups on top of 1.00 reads).

In the *hidden-source dispatch* suite the tasks are similar, but the source is not provided in the prompt. To avoid an initial whole-file read the use-site line and column are provided, but the type is no longer free in context and has to be fetched by read, grep or lookup. This suite ran three arms: text search alone (grep_base), lookup available but unprompted (defn_avail), and lookup with an instruction to use it (defn_prompt).

Arm	Resolved	Difference vs grep	Greps	Whole-file reads
text search	70/75	—	1.52	2.17
lookup available	71/75	+0.013 [−0.040, +0.067]	1.47	2.04
lookup prompted	75/75	+0.067 [+0.013, +0.133]	0.36	0.68

Per-task resolution over 5 seeds at temperature 0.7; 95% task bootstrap intervals over 15 tasks; greps and reads are per-task means. In the matched visible control every arm resolved all 75 rollouts.

Availability raised resolution on 3 but lowered it on 2 tasks. Prompting raised the resolution rate on 4 tasks with no impact on others. Tokens barely moved (0.983 prompted, 1.005 available, on tasks both arms solved).

We also verified that types are beneficial regardless of the tool as part of the dispatch ladder. Removing annotations hurt accurate retrieval, while the text agent succeeded 14-15/15 if the annotations were present at call site, construction site, or behind indirection.

3.2 FORM: does a compact span cost less than reading the file?

In the *whole-file baseline* suite, the whole-file arm cost 3.35x the definition arm in total tokens for Qwen3.6-27B, 3.49x for Sonnet 4.5 and 4.12x for DeepSeek v3.1. A capable agent rarely reads a whole file though.

The *text-retrieval baseline* suite explored the same 11 tasks of real library source. Qwen3.5-27B spent 1,602 mean tokens with text retrieval and 1,235 with definitions, a paired ratio of 1.297 [1.093, 1.527]. Definitions were cheaper on 10 of 11 and both arms solved each task.

In order to benefit from this efficiency gain, we need to establish whether the tool call augmented or substituted existing file reads. This behavior appears to be workload-dependent: in the *definition spans* suite, injected defn spans were followed by a read of the defining file on 35 of 36 instances. Prompting that the span was complete removed just 2 of those reads, while adding a per-task cost of 294 tokens.

Model	Reread, injected span	Reread, plus sufficiency instruction
Qwen3.6-27B	11/12	12/12
Sonnet 4.5	12/12	12/12
DeepSeek v3.1	12/12	10/12

The *shell-agent case study* showed similar results on real repositories: a manual cross-check matched 16 of 18 definition calls to a later read of the file just resolved.

Training can impact substitution. A separate DAgger-style relabel [8] on 39 demonstrations, none of which required a read, cut the reread from 11 of 12 to 0 of 12 on held-out instances with different seeds and templates. Mean input fell from 1,157 to 748 tokens, and from 1,493 to 938 on the nine tasks both arms solved, a 1.59x saving. Held-out pass slipped from 11 to 10 of 12, one rescue against two losses, which at 12 instances and one seed is indistinguishable from noise. We held back a further 12 instances, unused until the end: there the reread went from 12 of 12 to 1 of 12, mean input from 1,223 to 729 tokens, a 1.72x saving on the eleven both arms passed, and held-out pass from 12 to 11 of 12.

We also used the *definition spans* suite to test sufficiency: whether the model reads after a defn span is delivered if the information contained is insufficient. A model that did not read would gain token efficiency at the cost of correctness.

Reads after span	Span insufficient	Span sufficient	Helper also delivered
Untrained	12/12	12/12	11/12
Trained	12/12	2/12	0/12

The trained model went looking every time the span lacked the defect and largely stopped when it did not; the untrained model read in all three situations. Filtering to cases where reading was required, the trained model read only once against the untrained model’s 3.42 times. Both arms successfully completed all tasks.

3.3 WHEN: does the moment the diagnostic arrives change the outcome?

In the *checker grid* the model was given a draft change that passes its visible test but fails a held-out one, and asked to review it and submit. We compared injecting a diagnostic after every edit, one at revision, one that rejects a defective draft and asks for repair at submission time, and a no-checker control.

Left to itself, the model rarely touched the draft so the after-every-edit channel fired on only 1 of 12 defects and that arm matched the control at 1 of 12 accepted. One-shot delivery at revision reached 10 of 12, and the gate, which rejects a defective submission and asks for repair, reached 11 of 12. Its single miss was a repair that came back type-clean and still failed the held-out test.

12 seeded defect/clean pairs	Control	After every edit	One-shot	Gate
Bad completion accepted	11/12	11/12	2/12	1/12
Revision tokens, defect	787	754	1,210	1,380
Wall time, defect (s)	27.8	14.5	68.3	62.7
Revision tokens, clean	591	—	619	591
Wall time, clean (s)	9.4	9.9	9.6	9.5

Mean edits 0.08 against 0.17 in control.

Defects had to be seeded because capable models rarely left checker-detectable ones given the relatively simple tasks we were testing.

To test how the timing of diagnostics matters, the *delivery timing* suite put Qwen2.5-Coder-7B-Instruct and Pyrefly through six delivery points. The arms were no feedback, batched at yield, batched after each edit, and three variations of live channels that interrupt mid-generation, splicing the diagnostic into the stream as Hooper et al. [4] do for asynchronous tool results. The model was the stock instruct checkpoint in every arm.

Arm	When the diagnostic arrives	Resolved	Difference vs no feedback
none	never	81/168 = 0.482	—
lazy	batched, at the model’s next yield	89/168 = 0.530	+0.048 [−0.018, +0.113]
eager	batched, immediately after each edit	88/168 = 0.524	+0.042 [−0.024, +0.119]
naive	live mid-stream, told to fix each on arrival	57/168 = 0.339	−0.143 [−0.208, −0.077]
plain	live mid-stream	77/168 = 0.458	−0.024 [−0.113, +0.048]
gated	live mid-stream, only when the file parses	81/168 = 0.482	+0.000 [−0.054, +0.054]

14 tasks x 12 seeds = 168 paired rollouts per arm. Differences are task-clustered bootstraps; significance is paired exact McNemar under Benjamini-Hochberg FDR 5% across all 15 pairwise contrasts (cutoff $p \leq 0.0105$). Only the

naive arm's five contrasts clear it; among the other five arms the smallest p is 0.119. Arms are A, C-lazy, C-eager, D-naive, D-plain and D-gate in the run artifacts.

Batching at a yield, batching after every edit and interrupting mid-generation all land within noise of each other and of no feedback at all. What hurt was the instruction. The naive live arm differs from the plain live arm only by a sentence telling the model to fix each diagnostic before moving on, and removing that sentence recovers most of the deficit (+0.119 [+0.036, +0.202], $p=0.0105$). Most ungated diagnostics were about the model's own half-finished edit, so the channel is noisy as described. It is the standing order to act on every message, not the noise, that impacts the fix rate though.

4 Conclusions, if you use a coding agent

LSP services can aid coding agents, but require more consideration than simply making additional tools available.

Keep type annotations correct and present. Regardless of tools, having correct, visible types improved how agents navigated the source.

Run your type checker at the end of the agent's turn and make it blocking. This offered the clearest benefit and was the cheapest to wire; it costs no tokens on clean work. Count how often it blocks, how often the repair passes, and how often it rejects valid work.

Make navigation tools available and encourage their use for larger and more complex codebases. For code in context navigation tools add nothing, and for most well-typed code models will read and retrieve correct spans from the appropriate source. Code navigation tools offer the most gains where the target is somewhat obfuscated, as with generated stubs, compiled extensions, vendored packages, or objects assembled by a factory, registry or container.

Do not assume a tool saves tokens unless it substitutes for existing calls. When adding a definition tool you can end up still spending tokens on the ranged read: identify and measure duplicative invocations. Prompting may help, but deep changes to this behavior could require fine-tuning.

5 Future work

Retrieval calibration. Agents retrieve constantly and mostly indiscriminately: grep hits, file chunks, documentation, search results, other tools' output. If "do I have enough?" is trainable, it is a general lever on agent cost. We have seen tool-use post-training work well, and enable the model to discriminate on tool calls. Can we train to add additional code understanding channels, and to choose between them intelligently?

Tools co-designed with the agent. The tools tested were built for a human, who can ignore a transient diagnostic. An agent inherits those outputs, but results suggest they lack some of the discrimination in how to use them. Can a service better anticipate what an agent needs, suppress likely-unhelpful messages and give more directive feedback on actionable ones?

Assessing a codebase in advance. Results suggest types are beneficial to coding agents regardless of the availability of LSP tools, and adding LSP tooling offers wins primarily as codebases become more complex. It is difficult to evaluate whether a given code base will benefit from more aggressive usage of these types of tools however. Are there measures we can use to assess this and predict the benefit, such as annotation density, indirection depth between call and definition sites, or the share of types resolving only through a stub or compiled boundary?

Which tasks benefit. Everything here is dispatch ambiguity and small seeded defects in readable workspaces. Semantic tooling may matter in proportion to the sheer volume of context required, as seen in wide refactors, cross-module renames, API migrations. Can we create tasks and environments which evaluate these kinds of changes, and use them to identify useful policies for LSP-related tools?

6 Appendix: task construction and apparatus checks

Checker defects were screened so that each is coherent (the target file parses and leaves no unimplemented stub), passes the visible test, fails a held-out test, and carries a single semantic diagnostic in the target file; each clean control is that defect's own validated gold. The screens on the repository scan were environment, leakage, ambiguity and discriminating lookup.

In the sufficiency experiment, a scripted adversary searched 17 one-parameter repair forms and found none that reached the held-out test without retrieval, so the instances cannot be solved by guessing. The harness also re-shows any file the agent has edited, which would have delivered the hidden line without a read; we redacted that post-edit view while leaving deliberate reads and grep available. The training set behind that result contained no instances where a read was still required after the span, so the selectivity the trained model shows was never demonstrated to it; the result

establishes suppression of the post-span read rather than a learned conditional policy, and it was tested against a single form of insufficiency.

The static AST resolver behind the definition operation was checked against a live Pyrefly daemon at two levels: 12 of 12 agreement on the synthetic task symbols, and the same outcome with byte-identical input tokens on 22 paired real-library cells. That agreement covers definition lookup on workspaces of the kind used here, not the rest of the language-server surface. A further check on real library symbols agreed 9 of 11 with no disagreements, the two gaps being re-exports where Pyrefly returned null and the resolver did not, so the two are not interchangeable in general.

The shell-agent case study ran a single seed over 3 tasks with one model and a 60-step cap, under which only 2 of 9 arms converged. Its counts are a case series read alongside the controlled grids, not a matched comparison with them.

References

- [1] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. Iterative refinement of project-level code context for precise code generation with compiler feedback. In *Findings of the Association for Computational Linguistics*, 2024. CoCoGen; arXiv:2403.16792.
- [2] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2), 2024. arXiv:2409.00921.
- [3] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: LSP-guided RAG for language-agnostic real-time unit test generation. *arXiv preprint arXiv:2510.22210*, 2025.
- [4] Coleman Hooper, Minwoo Kang, Suhong Moon, Nicholas Lee, Eric Wen, John Wawrzynek, Michael W. Mahoney, Yakun Sophia Shao, Amir Gholami, and Kurt Keutzer. Speculative interaction agents: Building real-time agents with asynchronous I/O and speculative tool calling. *arXiv preprint arXiv:2605.13360*, 2026.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [6] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODE-STRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2026.
- [7] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting LLM-based repository-level code completion with static analysis. *arXiv preprint arXiv:2406.10018*, 2024.
- [8] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. PMLR v15:627–635.
- [9] SWE-agent Team. mini-swe-agent. <https://github.com/SWE-agent/mini-swe-agent>, 2025. Software, MIT license; built by the Princeton and Stanford team behind SWE-bench and SWE-agent.
- [10] Martin Vogel, Falk Meyer-Eschenbach, Severin Kohler, Elias Grünewald, and Felix Balzer. Codebase-memory: Tree-sitter-based knowledge graphs for LLM code exploration via MCP. *arXiv preprint arXiv:2603.27277*, 2026.
- [11] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.